sqıd: a Multipurpose Virtualization Tool, Streamlining Setups for UbiComp Research (Adjunct: Code Samples and Use Case Scenarios)

Roland Aigner
e-mail: rolandaigner@outlook.com

October 27, 2025

1 Code Samples

In the following, we show three exemplary how-tos that demonstrate how a practical utilization could look like. We do realize that the implementation of those pipelines are possibly way too basic for most realistic scenarios, however, to save space, the intention is not to provide robust methods, but to illustrate general *workflows* with sqıd. Operators are instantiated by selecting them in a context-menu or by typing (parts of) their names into a finder pop-up. Elements can be moved and patched by mouse control. Scenes are saved to JSON file, including all operator parameters and visualizer settings.

1.1 Array of Custom Resistive Sensors

We used four ADCs of an ESP32 Feather board to sample resistive strain sensors via a voltage divider each (cf. Figure 1a, left) with reference resistor $R_{ref} = 150\,\Omega$. We were only interested in relative values and actuation in this case, therefore we did *not* perform calculation or conversion to actual electrical units like Ohms or Volts. Instead, our objective was to reasonably map the sensor range to an output value range of [0 1], with 0 representing inactivity and 1 representing full actuation/saturation. Since the board features 12-bit ADCs, we normalized the ADC samples by scaling with $1/(2^{12}-1)$ and send the resulting four floating

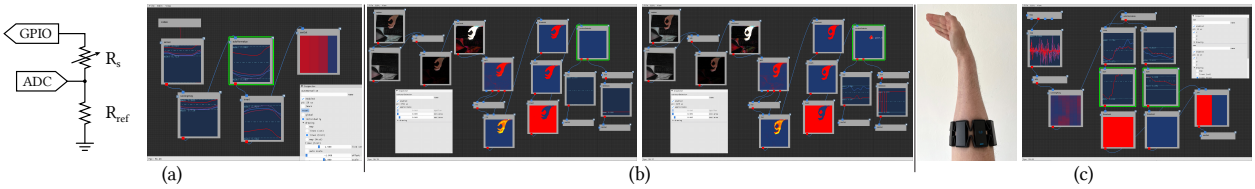

Figure 1: In three how-tos we sketch basic sqıd workflows. (a) we sampled four custom sensors (R_S) using ADCs of an ESP32 Feather board via voltage dividers. The data was received via serial port, filtered, values were re-mapped, inverted, and the result was sent for utilization via OSC protocol. (b) we implemented a TAFFI [1] pinch gesture detection from web-cam live imagery. Resulting data from blob-tracking (x/y-coordinates as well as binary on/off, see final operators on the right screenshot) were again sent to individual OSC destinations. (c) using a Thalmic Labs Myo 8-channel EMG armband, we implemented basic detection of hand bend direction by filtering, selecting and thresholding values of respective channels.

point values as a float-array via serial (COM4). Our sqid scene is shown in Figure 1a: a serial source operator was configured to receive data on COM4 (upper left corner), and forwarded data to a `sensor` node. Note the line-plot visualization that shows four individual trends of temporal sensor data which were slightly noisy due to the nature of our sensors. We applied an exponential smoothing filter with $\alpha = 0.1$ as a slight low-pass filter, which is implemented in the `runningAvrg` operator. Since the ADC sample values did not range from 4,095 down to 0 due to R_{ref}, we re-mapped the value input range. We did this with an `autoNormalize` operator, which can be used to find maxima and minima of individual slots for calibration, using the `learn` checkbox (cf. Inspector window), and hence normalizes input values according to the extrema that were observed during the "learning" period. Since sensor values of resistive sensors `drop` when actuated, the trend was upside-down and needed to be flipped to yield the signal we wanted, which we accomplished by an `invert` operator. The result was then sent to a UDP socket using OSC protocol, via an `oscOut` operator, for a potential receiving target application to pick up the data. Note the oscOut node's visualizer is set to "map", which renders a value map of most recent data instead of the line plots, which show a history of a configurable number of past samples.

1.2 TAFFI Web-Cam Based Pinch Detector

Based on the Thumb and Fore-Finger Interface (TAFFI) by Wilson [1], we demonstrate how to implement a very basic gesture pinch detection (on/off) including x/y coordinate control from web-camera live imagery (cf. Figure 1b). We started with a `capture` operator, to get USB camera input in RGB format. We used background subtraction in RGB-space to isolate the hand pixels as foreground area. While simple, *ad-hoc* background-models include adaptive methods, e.g., using temporal smoothing, we manually recorded a still image with a `s+h` (sample-and-hold) operator, for sake of simplicity. We subtracted this background image from the live camera data with a `sub` operator and took absolute values of the result with `abs`. We scaled the foreground values by a manually chosen constant factor of 6.75 to get good saturation of the remaining pixels using a `mulConst` operator. To amplify white (saturated) pixels and dampen non-white pixels, we multiplied all color channels, by splitting to RGB layers using a `split` operator, which slices the input 3D-matrix of dimension $A \times B \times C$ as C 2D-matrices of dimensions $A \times B$. We multiplied channels R and G using a `mul` operator and the result again with channel B using another `mul` node. To get a binary representation, we used a `threshold` at a manually adjusted value of $t = 0.8$. To get rid of salt noise, we applied a morphological opening filter with 2 iterations, provided in the `morph` operator. We now used the operator `contourDetector`, which is designed to perform closed component analysis, with a specified minimum and maximum area for size-based discarding of unwanted blobs. Since the contour detector treats zeros as background and ones as foreground, we first had to `invert` the input. The contour detector operator provides arrays of x/y pairs, representing the blobs' centroids on the first output pin, and a binary image containing the remaining blobs on the second pin. We used the second to implement an on/off switch, by summing all image values with a `sum` operator along all dimensions, and set a `threshold` of $t > 0$ to detect if there were any areas found at all. The output was sent via OSC using an `oscOut` sink, with a specific osc address pattern set to `/switch`. The x/y coordinates were sent via another `oscOut` with address pattern `/xy`, so both messages can be easily discerned by a receiving application, based on the OSC address. Note that for mere visualization purposes, we added a `nop` (no operation) to get line plots of the trends of x and y values.

1.3 Hand Bend Direction Detection Using Myo EMG Armband

Using the Thalmic Myo gesture armband featuring 8 EMG sensors, we demonstrate a straightforward classification of hand bend direction: from the `myo` source operator, we received 8-channel EMG data that we plotted using a `nop` node for visual inspection. We took absolute values of the waveforms with an `abs` operator, applied a slight exponential smoothing filter ($\alpha = 0.05$) to get rid of high-frequency noise with a

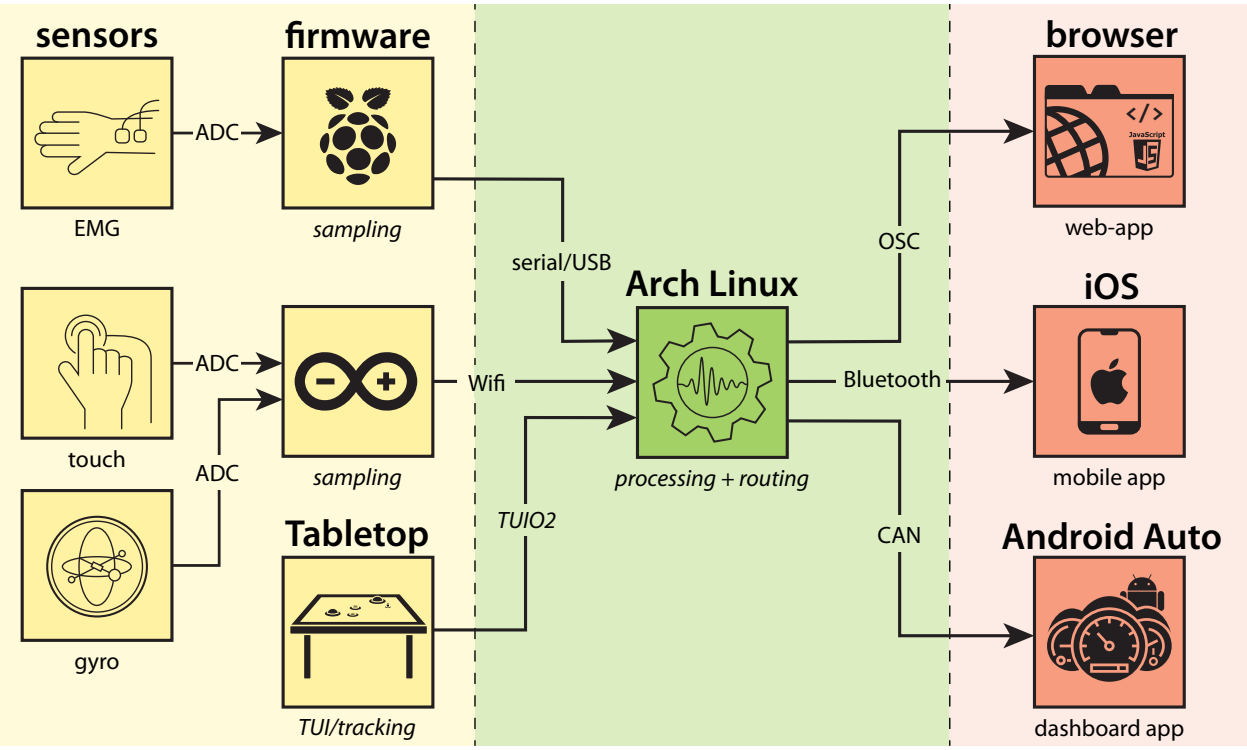

Figure 2: Hypothetical example setup, for demonstrating modular multi-component architecture: data from two MCUs (each sampling ≥1 sensors) and one TUIO host is processed and fused, resulting high-level command & control data is sent to respective endpoint applications, ran on diverse architectures.

`runningAvrg` operator. We then normalized individual sensor value ranges using an `autoNormalize`, which we calibrated for minima and maxima with a few seconds of random input data. We then split the 8 channel data into two 2-channel segments, using two `crop` operators. We isolated channels 3 and 4 (left) and channels 7 and 8 (right), and added those pairs together with a `sum` operator. We then used two `threshold` (t = 0.5) operators for left and right, respectively. We joined both values into a single 2-value frame with a `join` operator and sent the result using an `oscOut` to a UDP socket for potential utilization.

2 Usage as a Processing Hub

Figure 2 shows an exemplary setup using a single processing instance that connects multiple sources with multiple sinks. Note that input does not necessarily have to come from MCUs, which is illustrated by including a tabletop PC utilizing TUIO2[1]. Furthermore, our node-based VP interface supports an arbitrary number of sources and sinks in parallel, each of those using their individual physical interface and protocol. Consequently, sqid is able to perform sensor fusion, by merging data from multiple sources.

[1] `https://www.tuio.org/`

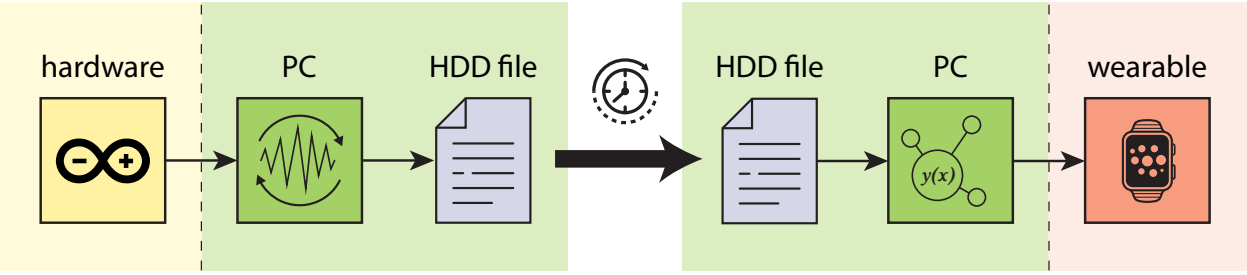

Figure 3: The feature of capturing timestamped live-data (either raw or pre-processed) to files on HDD for later replay as offline-data is intended to enable objective evaluation of pipeline/parameter modifications, and moreover provides reproducibility.

3 Usage as an Evaluation Tool

Besides utilization in concert with live sensors and devices, sqıd provides generator operators that output test data. Obvious candidates are white noise to simulate measurement noise, Perlin noise [2] to simulate basic natural-seeming input/movement, sinusoidal and non-sinusoidal waveforms to simulate periodic input. Moreover, we are able to write timed data to hard drive in binary, CSV, and MATLAB format using `fileOut` operators. These files can be replayed by `fileIn` operators, to either simulate input of devices currently not available, or to systematically test pipeline configurations (cf. Figure 3). Replay can be performed either timed, i.e., considering recorded timestamp-data, or disregarding timing. The latter is supposed to provide a batch execution mode, as opposed to the usual interactive mode [3], and can be useful to provide consistent input for controlled evaluation of various pipeline configurations. By running those sequentially, e.g., using a shell script, we are able to objectively compare their performances based on the computed results, which can be again be captured in CSV files and analyzed by the researchers using appropriate tools.

4 Case Studies

Taking our system to the test in the wild, we let a group of 5 researchers from the field of sensing, electronics, HCI, and UX design implement multiple demos and applications in subgroups. The group was part of our local institute, which enabled close collaboration and communication. The projects' nature was partly academic and partly contracted industry research.

4.1 Multi-Component Demo for Wearable Device: Smartwatch Controlled by Sensor-Strap

One team implemented a 4×5 matrix of capacitive sensors [4] on a smartwatch strap and used it to operate a watchOS app using micro-gestures (cf. Figure 4a). For demo purposes, the sensors were simply wired to a Cypress PSoC4 board for capacitive sensing. Raw data was sent via USB port to sqıd for processing which consisted mainly of interpolation and thresholding operations, blob detection, and calculation of the blob's center-of-mass for retrieving X and Y coordinates and relative blob area A, representing Z. Via Bluetooth, normalized XYZ-vectors were sent to a custom-built watchOS app that controlled a map according to these motion vectors.

In a similar setup, a demo was implemented for gesture detection based on drawing of strokes on the watch-strap. Again, the PSoC4 sent raw data that provided XYZ-coordinates via blob-tracking, which were used to draw strokes on a canvas in the watchOS app. In parallel, a $1 Unistroke Recognizer [5] C#

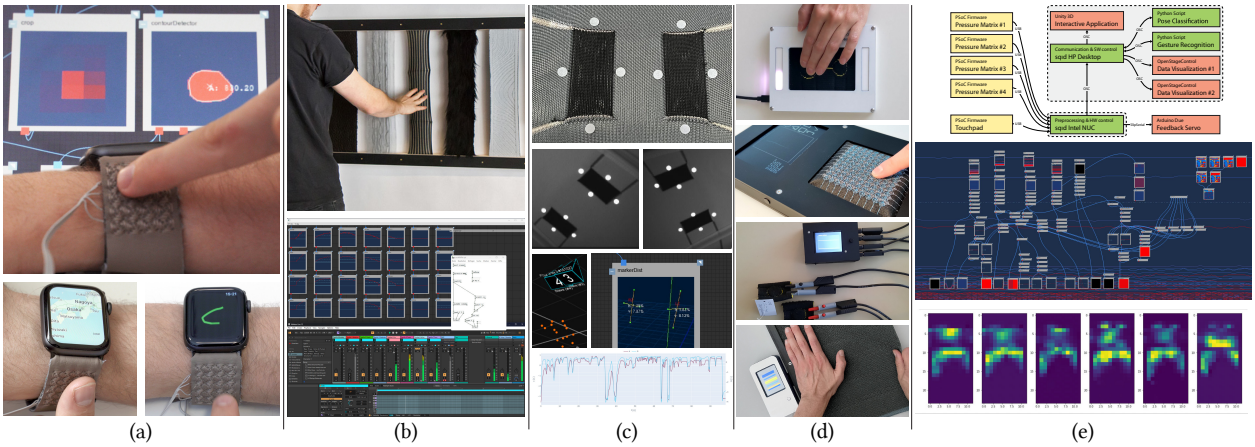

Figure 4: Impressions from the five case studies, included in this document: (a) a textile capacitive sensing matrix was implemented on a smartwatch strap for microgesture interaction. Raw data was filtered and interpolated for blob-tracking (X/Y position + area) that served as input for a watchOS app and a unistroke gesture recognizer. (b) several inputs from interactive fabrics in an art installation were pre-processed and forwarded to Pure Data, to trigger MIDI commands that controlled audio synthesizers in Ableton Live. (c) OptiTrack's IR-based optical marker tracking tool Motive supports streaming of marker positions via its NatNet networking protocol. An operator was implemented to receive those data and capture it along raw sensor data of knitted strain sensors. The recordings of marker data were utilized as ground-truth data for training ML-based alignment of faulty strain sensor data. (d) monolithic sensor demonstrators were built based on embedded devices, with data processing methods built directly into the firmware which were developed, tuned, and tested within sqid beforehand. (e) a complex multi-component installation was developed, ultimately using two instances of sqid that operated several devices and applications, while they provided an accessible interface for rapid calibration and troubleshooting.

desktop application received the same data via OSC. When the gesture was completed (on release, detected when $Z = A = 0$), the recognizer was triggered. On detection of a character, an according OSC package was returned to sqid and forwarded to the smartwatch via Bluetooth.

4.2 Multi-Component Interactive Audio-Installation

Another demo was implemented as an interactive audio-installation [6], interfacing multiple textile sensors, sampled by a Cypress PSoC4, with Pure Data and Ableton Live (cf. Figure 4b). Again, raw data was received by sqid via USB. The team applied several pre-processing steps, including exponential smoothing for de-noising, and value normalization. Ableton Live received MIDI commands, triggering and controlling audio effects. The team decided to not implement MIDI-functionality as an addon, instead it used a simple Pure Data patch to receive OSC messages that were simply translated to according MIDI commands.

4.3 Syncing of Multiple Sources for Capturing Data for Training Machine Learning Models

Using an automated testing setup for strain sensors, similar to [7] and [8], one team captured data to train machine learning models to infer strain from samples of knitted resistive sensors (cf. Figure 4c). Therefore, they captured a vast amount of raw sensor data along with ground-truth data that they calculated from

optically tracked markers, using a 6-camera OptiTrack system along with sqıd's NatNet extension. Live marker XYZ-positions were streamed via NatNet UDP interface into sqıd for calculation of strain based on marker distance. In parallel, they received raw resistive measurements from an ESP32 Feather Board via USB, which were synced with the marker position data and, along with a timestamp, captured to a CSV file for offline processing and training of machine learning algorithms in Juypter.

4.4 Prototyping for Tuning and Subsequent Ports to Self-Contained Embedded Devices

One team decided do build self-contained, autonomous devices as demonstrators for custom-made textile sensors, similar to [9] and [10]. They first developed, evaluated, and tuned the necessary algorithms for calibration, filtering of measurement noise, compensation of long-term drift and settling effects, gesture recognition, etc. When the processing pipeline was finished and proved reliable, they ported them unaltered to the devices' (Cypress PSoC4, NodeMCU, and Arduino) firmware C/C++ code and thus removed the prior dependency on an external host (cf. Figure 4d).

4.5 Multi-Modal Input: Combining Resistive Pressure Matrix with Touchpad Interaction

Combining four distributed pressure sensing matrices similar to [11] with a touch-pad, one project team built a complex multi-media installation including sophisticated data visualization and an interactive experience that allowed the user to control an interactive application using multi-modal input. The four matrices and one touch-pad were operated by a total of five PSoC4 boards that sent raw sample data via USB to sqıd, which was running on an Intel NUC mini-PC, for tasks like calibration, filtering, normalization, and blob-tracking, and furthermore to control an Arduino Due powered motor controller providing force-feedback, by sending OSC via SLIPSerial. sqıd furthermore communicated with a second instance by OSC via UDP, which was running on a desktop Windows PC and in turn interacted with two Python scripts that performed live gesture recognition on the touch-pad input data as well as body pose classification based on the pressure data (cf. Figure 4e). Moreover, sqıd controlled a co-located Unity3D interactive application as well as two instances of sensor data visualizations, both developed and running in Open Stage Control. All applications and Python environments on the desktop PC communicated using OSC via local UDP sockets. The Python sklearn classifiers were pre-trained with large amounts of sensor data that was previously captured to CSV files using sqıd.

This complex installation represents the most comprehensive utilization of sqıd to date. After completion, the researchers were asked about their experience and stated the project demonstrated the tool's versatility and scalability "particularly well". In detail, they stated that...

- ...the core data processing capabilities were already sufficient to filter, normalize, and map incoming data, merge multiple data matrices, and split parts of the data for individual processing and feature extraction using the linked Python environments.
- ...the networking interface made sqıd "useful for inter-connecting and orchestrating multiple entities and to relay data accordingly," which "facilitated a decentralized architecture" and "enabled a convenient operation."
- ...the feature for capturing data to CSV files was used "extensively for gathering training data for machine learning" purposes. Furthermore, since the already available recorded data could be replayed within the tool, "fine-tuning and testing was greatly simplified, since the effects of changes in parameterization could be verified instantly," which was "of great value."

References

[1] A. D. Wilson, "Robust computer vision-based detection of pinching for one and two-handed gesture input," in *UIST '06*. New York, NY, USA: ACM, 2006, p. 255–258. [Online]. Available: https://doi.org/10.1145/1166253.1166292

[2] K. Perlin, "An image synthesizer," in *SIGGRAPH '85*. New York, NY, USA: Association for Computing Machinery, 1985, p. 287–296. [Online]. Available: https://doi.org/10.1145/325334.325247

[3] B. A. Myers, "Visual programming, programming by example, and program visualization: a taxonomy," in *CHI '86*. New York, NY, USA: ACM, 1986, p. 59–66. [Online]. Available: https://doi.org/10.1145/22627.22349

[4] R. Aigner, A. Pointner, T. Preindl, R. Danner, and M. Haller, "Texyz: Embroidering enameled wires for three degree-of-freedom mutual capacitive sensing," in *CHI '21*. New York, NY, USA: ACM, 2021. [Online]. Available: https://doi.org/10.1145/3411764.3445479

[5] J. O. Wobbrock, A. D. Wilson, and Y. Li, "Gestures without libraries, toolkits or training: a $1 recognizer for user interface prototypes," in *UIST '07*. New York, NY, USA: ACM, 2007, p. 159–168. [Online]. Available: https://doi.org/10.1145/1294211.1294238

[6] S. Mlakar, T. Preindl, A. Pointner, M. Alida Haberfellner, R. Danner, R. Aigner, and M. Haller, "The sound of textile: An interactive tactile-sonic installation," in *ARTECH '21*. New York, NY, USA: ACM, 2022. [Online]. Available: https://doi.org/10.1145/3483529.3483742

[7] R. Aigner and A. Stöckl, "Machine-learning-based compensation for inconsistencies in knitted force sensors," *IEEE JSEN*, vol. 24, no. 4, pp. 4899–4906, 2024.

[8] S. Zühlke, A. Stöckl, and D. Schedl, "Touch sensing on semi-elastic textiles with border-based sensors," in *IHSED '23*, W. Karwowski, T. Ahram, M. Milicevic, D. Etinger, and K. Zubrinic, Eds., vol. 112. USA: AHFE International, 2023. [Online]. Available: https://doi.org/10.54941/ahfe1004106

[9] A. Pointner, T. Preindl, S. Mlakar, R. Aigner, and M. Haller, "Knitted RESi: A highly flexible, force-sensitive knitted textile based on resistive yarns," in *SIGGRAPH '20 ETech*. New York, NY, USA: ACM, 2020. [Online]. Available: https://doi.org/10.1145/3388534.3407292

[10] A. Pointner, T. Preindl, S. Mlakar, R. Aigner, M. A. Haberfellner, and M. Haller, "Knitted force sensors," in *UIST '22 Adjunct*. New York, NY, USA: ACM, 2022. [Online]. Available: https://doi.org/10.1145/3526114.3558656

[11] P. Parzer, K. Probst, T. Babic, C. Rendl, A. Vogl, A. Olwal, and M. Haller, "FlexTiles: A flexible, stretchable, formable, pressure-sensitive, tactile input sensor," in *CHI EA '16*. New York, NY, USA: ACM, 2016, p. 3754–3757. [Online]. Available: https://doi.org/10.1145/2851581.2890253